

班級：資訊三乙 學號：D1210799 姓名：王建葦

一、【實驗目的】：

What was your design? What were the concepts you have used for your design?

本次實驗的目的為熟悉 **UART 非同步序列通訊** 的實作方式，並結合 **Keypad**、**LCD**、**七段顯示器**、**Timer** 與**中斷機制**，完成一個雙板互動的 **1A2B 猜數字遊戲**。

本實驗需兩塊 NUC140 開發板以 UART 連線進行通訊，每一塊板皆同時扮演「出題者」與「猜題者」的角色，實作即時資料交換與結果回傳。

Lab12-1：1A2B 猜數字（UART 雙向通訊）

整體設計流程如下：

1. 隨機產生答案

- 系統啟動後，利用 `rand()` 產生 4 個 **0-9** 且不重複的數字
- 答案顯示於本地端七段顯示器（僅供自己觀看）

2. 輸入猜測數字

- 使用 **Keypad** 輸入 4 個數字
- 透過 **Timer0 中斷 (50Hz)** 掃描 Keypad
- 採用「按鍵狀態轉換偵測」，避免長按產生重複輸入

3. UART 傳送資料

- 當輸入滿 4 碼後，系統自動透過 **UART1 (115200 bps)** 將猜測數字送至對方開發板
- 使用 **UART TX Empty** 檢查，確保資料正確送出

4. UART 接收與判斷

- 透過 **UART1 接收中斷 (RDA Interrupt)** 接收對方猜測
- 收滿 4 碼後進行 **1A2B 計算**
 - A：數字與位置皆正確
 - B：數字正確但位置錯誤

5. 結果顯示與紀錄

- 將猜測結果格式化為：
1234 1A2B
- 顯示於 LCD
- LCD 最多顯示 **最近 4 次紀錄**，超過則以上捲方式更新

本實驗核心概念包含：

- UART 非同步通訊
- 中斷式資料接收

- Timer 驅動 Keypad 掃描
- 1A2B 邏輯演算法
- 多工任務 (Keypad / UART RX / LCD / 7-Segment)

二、【遭遇的問題】：

What problems you faced during design and implementation?

1. Keypad 長按時會重複輸入相同數字，導致猜測數字錯誤。
2. UART 接收資料若未妥善處理，可能出現資料錯位或遺失。
3. LCD 顯示多行歷史紀錄時，畫面更新容易閃爍。
4. 七段顯示器若未使用掃描機制，亮度不均且容易閃爍。
5. 亂數產生的答案若未檢查重複，會違反 1A2B 規則。
6. 同時處理 Keypad、UART 與顯示時，主迴圈容易變得混亂。

三、【解決方法】：

How did you solve the problems?

1. 在 **Timer0 IRQ** 中記錄前一次按鍵狀態，只在「未按 → 按下」時才註冊按鍵，成功避免連續輸入問題。
2. 使用 **UART RX 中斷**，並搭配接收索引 `g_i8RX_Index`，確保每次完整接收 4 筆資料後才進行判斷。
3. 將 LCD 顯示封裝為 `Add_Result_To_History()`，集中處理清屏與重繪，降低閃爍。
4. 利用 **Timer1 (200Hz)** 進行七段顯示器多工掃描，使顯示穩定且亮度一致。
5. 在 `Generate_Secret()` 中加入重複檢查迴圈，確保答案四碼皆不相同。
6. 以旗標 (Flag) 方式分離任務：
 - Keypad 輸入
 - UART 接收完成
 - 結果計算與顯示
 提升程式可讀性與穩定性。

在找尋這些問題的解決方法與問題點時，我有使用 ChatGPT 協助我找尋與解決問題。包

含 實驗結報的內容修改與潤飾都有使用 ChatGPT 協助。

四、【未能解決的問題】：

Was there any problem that you were unable to solve? Why was it unsolvable?

- 若 UART 線路干擾或接觸不良，仍可能造成通訊錯誤，需硬體層改善。
- 隨機數使用 `rand()`，未使用真實時間作為種子，重啟後答案可能重複。
- LCD 為單緩衝顯示，快速更新時仍有輕微閃爍現象。
- 目前遊戲未設計勝利條件（如 4A 結束遊戲），僅顯示結果。
- 無法處理錯誤輸入（如重複數字），僅依題目規格實作基本功能。

五、【程式碼】：

Main.c:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include "NUC100Series.h"
5  #include "MCU_init.h"
6  #include "SYS_init.h"
7  #include "LCD.h"
8  #include "Scankey.h"
9  #include "Seven_Segment.h"
10
11 // --- Global Variables ---
12 volatile uint8_t secret[4] = {0,0,0,0};
13 volatile uint8_t guess_buf[4];
14 volatile uint8_t opponent_guess[4];
15
16 volatile uint8_t g_u8Key_Flag = 0;
17 volatile int g_i8Key_Index = 0;
18 volatile int g_i8RX_Index = 0;
19 volatile uint8_t g_u8RX_Complete = 0;
20
21 volatile uint32_t cnt_5ms = 0;
22
23 // --- History Buffers ---
24 char g_History[4][17] = {
25     "          ",
26     "          ",
27     "          ",
28     "          "
29 };
30
31 // --- Function Prototypes ---
32 extern void print_Line(uint8_t line, char text[]);
33 extern void init_LCD(void);
34 extern void clear_LCD(void);
35
36 void Init_Timer0(void);
37 void Init_Timer1(void);
38 void Init_UART1(void);
39 void Generate_Secret(void);
40 void Calculate_1A2B(volatile uint8_t *secret_arr, volatile uint8_t *guess_arr, int *A, int *B);
41 void Add_Result_To_History(volatile uint8_t *guess, int A, int B);

```

```

43 int main(void)
44 {
45     int A, B;
46     uint32_t i;
47
48     // 1. Initialization
49     SYS_Init();
50     Init_Timer0();
51     Init_Timer1();
52     Init_UART1();
53
54     init_LCD();
55     clear_LCD();
56     OpenKeyPad();
57
58     // 2. Game Setup
59     Generate_Secret();
60
61     while(1)
62     {
63         // --- Task A: Process Keypad Input ---
64         if (g_u8Key_Flag != 0)
65         {
66             if (g_i8Key_Index < 4) {
67                 guess_buf[g_i8Key_Index] = g_u8Key_Flag;
68                 g_i8Key_Index++;
69             }
70
71             // If 4 digits entered, Send to Opponent
72             if (g_i8Key_Index == 4) {
73                 for(i=0; i<4; i++) {
74                     while((UART1->FSR & UART_FSR_TX_EMPTY_Msk) == 0);
75                     UART_WRITE(UART1, guess_buf[i]);
76                 }
77
78                 g_i8Key_Index = 0;
79             }
80
81             g_u8Key_Flag = 0;
82         }
83
84         // --- Task B: Process Received Guess ---
85         if (g_u8RX_Complete == 1)
86         {
87             Calculate_1A2B(secret, opponent_guess, &A, &B);
88             Add_Result_To_History(opponent_guess, A, B);
89             g_u8RX_Complete = 0;
90         }
91     }
92 }

```

```

97 void TMR0_IRQHandler(void)
98 {
99     static uint8_t u8LastKey = 0; // Remembers previous state
100     uint8_t u8CurrentKey;
101
102     TIMER_ClearIntFlag(TIMER0);
103
104     u8CurrentKey = ScanKey();
105
106     // logic: Only register if Key is pressed NOW, but was NOT pressed before.
107     // This prevents "1111" when holding the button.
108     if (u8CurrentKey != 0 && u8LastKey == 0) {
109         g_u8Key_Flag = u8CurrentKey;
110     }
111
112     u8LastKey = u8CurrentKey; // Update history
113 }
114
115 void TMR1_IRQHandler(void)
116 {
117     TIMER_ClearIntFlag(TIMER1);
118     cnt_5ms++;
119
120     if (cnt_5ms % 4 == 0) {
121         CloseSevenSegment();
122         ShowSevenSegment(3, secret[0]);
123     }
124     else if (cnt_5ms % 4 == 1) {
125         CloseSevenSegment();
126         ShowSevenSegment(2, secret[1]);
127     }
128     else if (cnt_5ms % 4 == 2) {
129         CloseSevenSegment();
130         ShowSevenSegment(1, secret[2]);
131     }
132     else if (cnt_5ms % 4 == 3) {
133         CloseSevenSegment();
134         ShowSevenSegment(0, secret[3]);
135     }
136 }

```

```

138 void UART1_IRQHandler(void)
139 {
140     uint32_t u32IntSts = UART1->ISR;
141     uint8_t c;
142
143     if(u32IntSts & UART_ISR_RDA_INT_Msk)
144     {
145         c = UART_READ(UART1);
146
147         if (g_i8RX_Index < 4) {
148             opponent_guess[g_i8RX_Index] = c;
149             g_i8RX_Index++;
150         }
151
152         if (g_i8RX_Index == 4) {
153             g_i8RX_Index = 0;
154             g_u8RX_Complete = 1;
155         }
156     }
157 }
158
159 // --- Initialization Functions ---
160
161 // UPDATED: Changed from 5Hz to 50Hz
162 void Init_Timer0(void) {
163     // 50Hz = Check keypad every 20ms (Much faster!)
164     TIMER_Open(TIMER0, TIMER_PERIODIC_MODE, 50);
165     TIMER_EnableInt(TIMER0);
166     NVIC_EnableIRQ(TMR0_IRQn);
167     TIMER_Start(TIMER0);
168 }
169
170 void Init_Timer1(void) {
171     TIMER_Open(TIMER1, TIMER_PERIODIC_MODE, 200);
172     TIMER_EnableInt(TIMER1);
173     NVIC_EnableIRQ(TMR1_IRQn);
174     TIMER_Start(TIMER1);
175 }
176
177 void Init_UART1(void) {
178     UART_Open(UART1, 115200);
179     UART_EnableInt(UART1, UART_IER_RDA_IEN_Msk);
180     NVIC_EnableIRQ(UART1_IRQn);
181 }

```

```

185 void Generate_Secret(void) {
186     int i, j;
187     int is_duplicate;
188
189     for (i = 0; i < 4; i++) {
190         do {
191             is_duplicate = 0;
192             secret[i] = rand() % 10;
193
194             for (j = 0; j < i; j++) {
195                 if (secret[i] == secret[j]) {
196                     is_duplicate = 1;
197                     break;
198                 }
199             }
200         } while (is_duplicate);
201     }
202 }
203
204 void Calculate_1A2B(volatile uint8_t *secret_arr, volatile uint8_t *guess_arr, int *A, int *B) {
205     int i, j;
206     *A = 0;
207     *B = 0;
208
209     for (i = 0; i < 4; i++) {
210         if (guess_arr[i] == secret_arr[i]) {
211             (*A)++;
212         } else {
213             for (j = 0; j < 4; j++) {
214                 if (guess_arr[i] == secret_arr[j]) {
215                     (*B)++;
216                     break;
217                 }
218             }
219         }
220     }
221 }
222
223 void Add_Result_To_History(volatile uint8_t *guess, int A, int B) {
224     strcpy(g_History[0], g_History[1]);
225     strcpy(g_History[1], g_History[2]);
226     strcpy(g_History[2], g_History[3]);
227
228     sprintf(g_History[3], "%d%d%d%d %dA%dB",
229             guess[0], guess[1], guess[2], guess[3], A, B);
230
231     clear_LCD();
232     print_Line(0, g_History[0]);
233     print_Line(1, g_History[1]);
234     print_Line(2, g_History[2]);
235     print_Line(3, g_History[3]);
236 }

```

MCU_init.h:

```
1 //Define Clock source
2 #define MCU_CLOCK_SOURCE
3 #define MCU_CLOCK_SOURCE_HXT // HXT, LXT, HIRC, LIRC
4 #define MCU_CLOCK_FREQUENCY 50000000 //Hz
5
6 //Define MCU Interfaces
7 #define MCU_INTERFACE_SPI3
8 #define SPI3_CLOCK_SOURCE_HCLK // HCLK, PLL
9 #define PIN_SPI3_SS0_PD8
10 #define PIN_SPI3_SCLK_PD9
11 #define PIN_SPI3_MISO0_PD10
12 #define PIN_SPI3_MOSI0_PD11
13
14 // ... Your existing SPI3 definitions ...
15
16 // --- Add these for Lab 12-1 (1A2B) ---
17
18 // 1. Enable UART1 for Communication
19 #define MCU_INTERFACE_UART1
20 #define UART_CLOCK_SOURCE_HXT // Use External Crystal (12MHz) for stable baud rate
21 #define UART_CLOCK_DIVIDER 1
22 #define PIN_UART1_RX_PB4 // Default RX Pin on NUC140 Learning Board
23 #define PIN_UART1_TX_PB5 // Default TX Pin on NUC140 Learning Board
24
25 // 2. Enable Timer0 for Keypad Scanning (200ms)
26 #define MCU_INTERFACE_TMR0
27 #define TMR0_CLOCK_SOURCE_HXT // Use HXT for accurate timing
28
29 // 3. Enable Timer1 for 7-Segment Multiplexing (5ms)
30 #define MCU_INTERFACE_TMR1
31 #define TMR1_CLOCK_SOURCE_HXT
```